



UNIVERSITÉ DU
LUXEMBOURG

FACULTY OF SCIENCE, TECHNOLOGY AND MEDICINE

VLA for E2E Autonomous Driving

Thesis Submitted in Partial Fulfillment of the
Requirements for the Degree of Master in
Information and Computer Sciences

Author:
Abdollah GADER

Supervisor:
Prof. Dr. Raphaël FRANK

Reviewer:
Prof. Fabrizio PASTORE

Advisor:
Dr. Gamal ELGHAZALY

September 2026

Abstract

End-to-end driving models predict a trajectory directly from sensor data, usually a single trajectory per scene. UniDriveVLA [1] is one such model. DrivoR [2] instead generates several candidate trajectories per scene and selects one with a learned scorer. This thesis tests whether DrivoR’s approach improves UniDriveVLA, which is trained only on nuScenes [3], and whether the combination reaches state-of-the-art results on the NAVSIM benchmark [4] without fine-tuning on NAVSIM data. We build an evaluation pipeline that scores UniDriveVLA’s trajectories with EPDMS, the driving score of NAVSIM v2 [5]. We add eight learned mode embeddings to UniDriveVLA’s action expert and train them, together with the action expert’s input and output layers, with a relaxed winner-takes-all objective. The rest of the model stays frozen. The model then produces eight candidate trajectories per scene. A separate scorer predicts the EPDMS of each candidate and selects the candidate with the highest prediction. The scorer is trained and evaluated on NAVSIM’s navhard split with five-fold cross-validation. On navhard, the model’s default mode reaches an EPDMS of 10.0. Selecting candidates with the scorer raises it to 12.7. Planners trained on NAVSIM score higher, for example DrivoR with 48.3. On 63 % of the scenes, the default mode scores zero. A zero score results from an at-fault collision, leaving the drivable area, driving against the direction of the lane, or running a red light. Learned candidate selection thus improves a vision–language–action model that was never trained on NAVSIM, but closing the gap to NAVSIM-trained planners requires adapting the model to NAVSIM data.

Keywords: End-to-End Autonomous Driving; Vision–Language–Action Models; Candidate Trajectory Selection; NAVSIM.

Acknowledgments

I would like to thank my supervisor, Prof. Dr. Raphaël Frank, for his guidance and support throughout this thesis. I am also grateful to my advisor, Dr. Gamal Elghazaly, for his continuous feedback, technical guidance, and patience throughout the challenges encountered during this work. Their support was essential to the completion of this thesis.

Contents

1	Introduction	1
2	Related Work	3
3	Methodology	6
3.1	Notation	6
3.2	Camera adaptation	6
3.3	UniDriveVLA experts	8
3.4	Trajectory generation with flow matching	9
3.5	Mode embeddings	10
3.6	Relaxed winner-takes-all training	10
3.7	Conversion to NAVSIM	12
3.8	EPDMS	12
3.9	Learned scorer	14
3.10	Inference	15
4	Experiments	17
4.1	Experimental setup	17
4.2	Evaluation metrics	18
4.3	Pipeline check	19
4.4	Diversity of the released model	20
4.5	Training the mode embeddings	20
4.6	Candidates on navhard	20
4.7	Learned scorer	21
4.8	Ablation of the scorer	22
5	Discussion	24
5.1	Interpretation of the results	24
5.2	Case studies	25
5.3	Limitations	26
5.4	Future work	27
5.5	Conclusion	27

List of Figures

3.1	System overview	7
3.2	Camera adaptation	7
3.3	Attention mask of masked joint attention	9
3.4	Trajectory generation with flow matching	10
3.5	Training of the mode embeddings	11
3.6	Conversion to NAVSIM	13
3.7	Two-stage scoring in NAVSIM v2	14
3.8	The learned scorer	15
4.1	Evaluation pipeline	18
4.2	Checkpoint selection for the mode training	20
4.3	Cross-validation of the scorer on navhard	21
4.4	Modes selected by the learned scorer	22
5.1	Case studies on navhard	26

List of Tables

3.1	Trained and frozen components	12
4.1	Training settings	18
4.2	Pipeline check on the warm-up split	19
4.3	Scores of the eight candidates on navhard	21
4.4	Selection policies on navhard	22
4.5	EPDMS on navhard	23
4.6	Ablation of the scorer on navhard	23
5.1	Design of the compared planners	25

Chapter 1

Introduction

A self-driving system has to decide, from its sensor data, where the vehicle goes next. Traditionally, this decision is made by a chain of separate modules: perception detects objects and lanes, prediction forecasts how these objects move, and planning computes the vehicle’s path. An *end-to-end planner* replaces this chain with one neural network that maps sensor data directly to a *trajectory*, the path the vehicle follows over the next few seconds [6]. Vision–language–action (VLA) models are a more recent type of end-to-end planner. They build on a pretrained vision–language model, a network trained on large amounts of images and text, and add a component that outputs a trajectory. UniDriveVLA [1] is such a model.

UniDriveVLA predicts one trajectory per scene. In many situations, for example at a junction or behind a slow vehicle, several manoeuvres are possible. A model with a single output cannot represent these alternatives or compare them. Hydra-MDP [7], DriveSuprim [8], and DrivoR [2] address this by generating several candidate trajectories per scene and selecting one with a learned scorer. These methods are developed and trained on NAVSIM [4], a benchmark that evaluates trajectories against recorded driving logs. Its second version, NAVSIM v2 [5], introduced the score EPDMS used in this thesis. To our knowledge, these candidate-selection methods have not been combined with a VLA model, nor with a model trained on a different dataset than the one it is evaluated on. UniDriveVLA is trained on nuScenes [3], not on NAVSIM. Retraining a model of this size for every new dataset is costly, so it is relevant whether candidate selection can improve it without such retraining.

This thesis therefore asks two research questions, for UniDriveVLA without training on NAVSIM data:

- Does selecting among candidate trajectories with a learned scorer, as in DrivoR, improve EPDMS over the model’s default trajectory?
- Does this combination reach results comparable to state-of-the-art planners on NAVSIM?

Contributions.

- An evaluation of UniDriveVLA on NAVSIM. To our knowledge, this is the first NAVSIM result for a VLA model trained on nuScenes (Chapter 4).

- An extension of UniDriveVLA that produces several candidate trajectories per scene with learned mode embeddings (Chapter 3).
- A learned scorer that selects among UniDriveVLA’s candidates (Chapters 3–4).

Scope. UniDriveVLA’s pretrained network stays frozen; only the mode embeddings and the action expert’s input and output layers are trained. Evaluation uses NAVSIM v2, which scores each scene in two stages. The first stage starts from the recorded scene. The second stage starts from synthetic positions of the ego vehicle around where the recorded driver was four seconds later, which tests whether the planner recovers from small deviations. In both stages, other vehicles react to the ego vehicle through a rule-based model. Closed-loop simulation is not part of this thesis.

Outline. Chapter 2 reviews end-to-end planning, the NAVSIM benchmark and EPDMS, methods that generate and score candidate trajectories, and VLA models for driving. Chapter 3 describes the adaptation of NAVSIM’s cameras to UniDriveVLA, the mode embeddings, and the scorer. Chapter 4 describes the evaluation pipeline and reports the results on NAVSIM. Chapter 5 interprets the results, compares them with existing planners, shows example scenes, and closes with the limitations, future work, and the conclusion.

Chapter 2

Related Work

Evaluation of driving planners. Planners are evaluated in one of three ways. Open-loop evaluation compares the predicted trajectory with the trajectory of the human driver, for example with the $L2$ distance and the collision rate on nuScenes [3]. It is cheap, but it rewards imitation rather than safe driving: a planner that only extrapolates the ego vehicle’s past motion already scores well [9]. Closed-loop evaluation lets the planner drive in a simulator. After each planned trajectory, the simulator moves the vehicle along the first part of it and updates the scene. The planner then predicts a new trajectory from the new state. Errors therefore accumulate as in real driving. This requires a simulator that renders new sensor data for every state, which is costly.

NAVSIM [4] lies between the two. The planner predicts one trajectory from the recorded sensor data of a scene and never sees the result of its own trajectory. NAVSIM then simulates this trajectory for four seconds in the recorded scene and scores it with rules, instead of comparing it with the human trajectory. NAVSIM v1 [4] uses the Predictive Driver Model Score (PDMS), derived from the rule-based PDM planner [9]. PDMS multiplies penalty terms for at-fault collisions and for leaving the drivable area with a weighted average of progress, time-to-collision, and comfort. In NAVSIM v1, other vehicles follow their recorded paths during the simulation. NAVSIM v2 [5] replaces PDMS with the Extended PDMS (EPDMS), which adds terms for driving direction, traffic lights, lane keeping, and extended comfort. In NAVSIM v2, other vehicles react to the ego vehicle through a rule-based model. NAVSIM v2 also scores each scene twice. The first score uses the recorded sensor data. The second uses synthetic camera images rendered at ego positions around where the human driver was four seconds later, and tests whether the planner recovers when it starts from a slightly different position. EPDMS combines both scores. Because NAVSIM computes these scores automatically, it can score any number of candidate trajectories. This makes it a source of training labels for learned scorers [7, 2].

Single-output end-to-end planners. Most end-to-end planners output one trajectory per scene. They differ mainly in how they represent the scene. UniAD [6] chains perception, prediction, and planning modules in one network trained for planning. It represents the scene as a dense grid of features seen from above. VAD [10] describes lanes and objects as vectors, that is, as lines and boxes, instead of grid cells, and reaches similar planning quality at lower cost. SparseDrive [11] does not build a grid at all. It uses a fixed number of learned vectors, called queries, each of which collects image

features for one object or one map element. UniDriveVLA’s perception reuses this design. TransFuser [12] fuses camera and LiDAR features with a transformer and is a common NAVSIM baseline. A single output is a limitation when a scene allows several manoeuvres. These planners are trained to minimise the distance to the one recorded trajectory of each scene. If similar scenes were recorded with different manoeuvres, for example overtaking a vehicle on the left in one recording and on the right in another, the trajectory with the smallest average distance lies between the two and leads into the vehicle [13]. Planners that output several trajectories avoid this problem.

Multimodal trajectory generation. Methods that output several trajectories per scene take one of two approaches. The first approach samples trajectories from a generative model. Diffusion models [14] and flow-matching models [15] both start from random values, drawn from a Gaussian distribution for each point of the trajectory, and transform them step by step into a trajectory. A diffusion model learns to predict and remove the noise in each step. A flow-matching model learns a velocity that moves the values along a straight path towards the trajectory. Different random starting values can give different trajectories. If the model is trained on one recorded trajectory per scene, different starting values often lead to nearly the same trajectory, an effect called mode collapse [16]. DiffusionDrive [16] reduces it by starting from fixed trajectories, which the authors call anchors, with added random values. The anchors are obtained by clustering the trajectories of the training set.

The second approach lets the network predict a fixed number N of trajectories for each scene. The n -th of these trajectories is called mode n . During training, only the mode whose trajectory is closest to the recorded trajectory receives the loss; the other modes are not updated for that scene. This training method is called winner-takes-all or multiple-choice learning [17] and is widely used in motion prediction [13]. Different modes are thus trained on different scenes and learn different manoeuvres, although each scene has only one recorded trajectory.

Selecting among candidate trajectories. Several planners generate a set of candidate trajectories and select one with a scorer. They differ in how the candidates are generated and how the scorer is built. PDM-Closed [9] generates candidates by following the centreline of the route at several target speeds and with several sideways offsets. It simulates each candidate and selects the best one according to the PDM rules, from which PDMS is derived. Its rules need the positions of other vehicles and the map from the dataset annotations, which a planner that only uses cameras does not have. VADv2 [18] and Hydra-MDP [7] select from a fixed set of candidate trajectories, called a vocabulary, obtained by clustering the trajectories of the training set. VADv2 predicts for each candidate the probability that it matches the human trajectory. Hydra-MDP predicts each candidate’s PDMS terms and is trained both on the distance to the human trajectory and on the terms computed by NAVSIM. DriveSuprim [8] scores the whole vocabulary, keeps the best candidates, and scores these again with a second, more detailed scorer. GTRS [19] adds candidates from a diffusion model to the vocabulary. ZTRS [20] trains its scorer only with rewards computed by NAVSIM’s scoring rules, without human trajectories. DrivoR [2] generates its candidates from learned queries, one per candidate, and predicts the EPDMS terms of each candidate. Because the terms are predicted separately, the weights that combine them can be changed after training, for example to give safety more weight than progress.

On navhard, the test split of NAVSIM v2, GTRS, ZTRS, and DrivoR reach an EPDMS of 45.0 to 48.3, against 23.1 for the single-output Latent TransFuser [5, 2]. These three methods share two properties. Their scorers are trained on scores computed by NAVSIM for the scenes of NAVSIM’s training split, and their perception networks are trained or fine-tuned on the same split. Their results therefore show what candidate selection achieves on the data the planner is trained on, not on data from another dataset.

Vision–language and vision–language–action models. Vision–language models (VLMs) are trained on large collections of images with text and can describe images and answer questions about them. DriveVLM [21] uses a VLM to describe the scene, analyse it, and output a rough plan. Its variant DriveVLM-Dual refines this plan with a planner that uses 3D object detections, as in a modular pipeline. VLA models add a component that outputs the trajectory directly. AutoVLA [22] encodes trajectories as discrete tokens that the language model generates like words. ReCogDrive [23] combines a VLM with a diffusion model that generates the trajectory and is trained and evaluated on NAVSIM. SimpleVSF [24] uses a VLM in a different role: as an additional scorer of candidates generated by other planners. VLA models have billions of parameters, which makes training them on each new dataset costly.

Transfer to new datasets. The planners above are trained and tested on data from the same dataset. Datasets differ in vehicles, cameras, cities, and traffic rules, and adapting a planner to a new city usually requires new recorded driving [25]. NAVSIM is built from nuPlan, which was recorded with different vehicles and cameras than nuScenes and partly in other cities.

UniDriveVLA. UniDriveVLA [1] is the planner this thesis builds on. It consists of three transformer models, called experts, that exchange information: an understanding expert based on the Qwen3-VL vision–language model [26], a perception expert that follows SparseDrive [11], and an action expert that generates six future positions at 0.5 s intervals with flow matching [15]. It is trained on nuScenes and outputs one trajectory per scene. Its authors evaluate it on nuScenes and in closed-loop simulation, but not on NAVSIM.

Research gap. The works above leave one combination untested. Candidate selection improves planners that are trained on NAVSIM. In DrivoR’s experiments on NAVSIM’s validation split, using 64 candidates instead of one raises PDMS from 80.1 to 90.0 [2]. ReCogDrive is trained on the dataset it is evaluated on, and UniDriveVLA outputs one trajectory. To our knowledge, it has not been tested whether candidate selection also improves a VLA model on a dataset it was not trained on, and how close such a model then comes to the planners trained on that dataset. Chapter 3 describes how this thesis tests it with UniDriveVLA.

Chapter 3

Methodology

This chapter describes the planner evaluated in this thesis (Figure 3.1). Its inputs are six camera images with their calibration, the ego vehicle’s recent positions, and a navigation command. UniDriveVLA [1] processes them with three transformer models, called experts. The action expert is run once for each of N learned mode embeddings and outputs N candidate trajectories. A learned scorer predicts the EPDMS terms of each candidate and selects one.

3.1 Notation

All coordinates are in the *ego frame*, which is fixed to the vehicle, with x pointing forward and y to the left. A *pose* is a position with a heading. The inputs are

- the images I_v of $V = 6$ cameras, $v = 1, \dots, V$, resized as described in Section 3.2;
- the ego history $\mathbf{h} \in \mathbb{R}^{T_h \times 2}$, the ego positions at the last $T_h = 4$ frames in the current ego frame;
- the navigation command $c \in \{\text{left}, \text{straight}, \text{right}\}$. NAVSIM’s command has a fourth value, *unknown*, which we map to *straight*.

The output is a trajectory $\tau = (\mathbf{p}_1, \dots, \mathbf{p}_T)$ of $T = 6$ future positions $\mathbf{p}_k \in \mathbb{R}^2$ at intervals of $\Delta t = 0.5$ s, a horizon of 3 s. With N modes, the planner outputs N candidates $\tau^{(0)}, \dots, \tau^{(N-1)}$.

3.2 Camera adaptation

The perception expert places learned queries in 3D space and projects them into each camera image to read image features. It therefore needs, for each camera, the matrix that maps a 3D point to image coordinates. UniDriveVLA expresses 3D points in the frame of the vehicle’s LiDAR sensor, as in nuScenes. For camera v , let $R_v \in SO(3)$ and $\mathbf{t}_v \in \mathbb{R}^3$ be the rotation and translation from the LiDAR frame to the camera frame (the *extrinsics*), and $K_v \in \mathbb{R}^{3 \times 3}$ the camera’s focal lengths and optical centre (the *intrinsics*). A point $\mathbf{X}$ in the LiDAR frame projects to image coordinates $\mathbf{u}$ as

$$\lambda \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = P_v \begin{bmatrix} \mathbf{X} \\ 1 \end{bmatrix}, \quad P_v = K_v [R_v \mid \mathbf{t}_v] \in \mathbb{R}^{3 \times 4}, \quad (3.1)$$

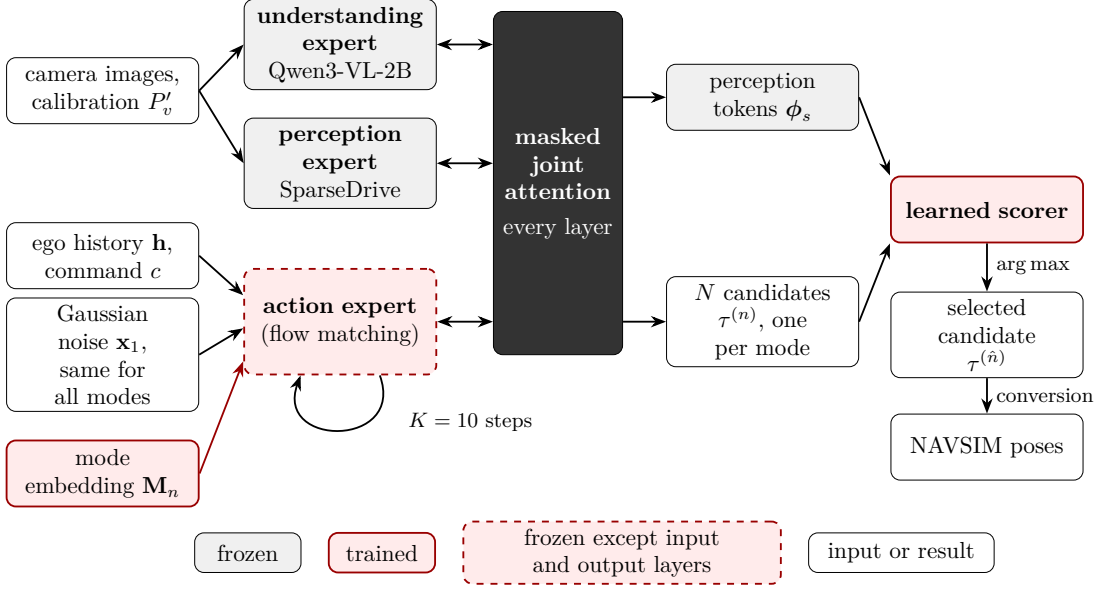


Figure 3.1: **System overview.** The understanding and perception experts read the camera images once per scene. The action expert turns Gaussian noise into a trajectory in $K = 10$ steps and is run once for each of the N mode embeddings. The three experts exchange information through masked joint attention. The learned scorer selects one candidate using the perception tokens, and the selected candidate is converted to NAVSIM poses.

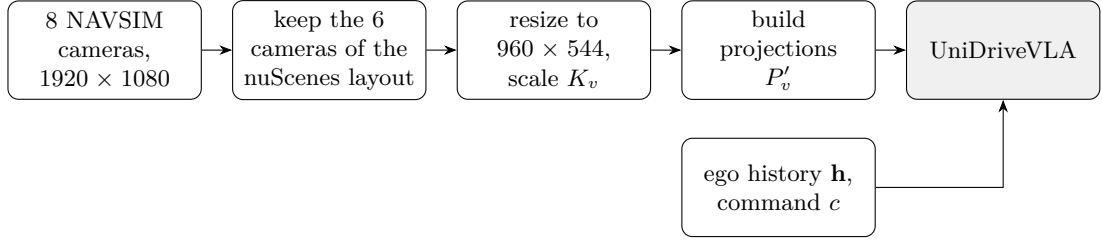


Figure 3.2: **Camera adaptation.** Six of NAVSIM’s eight cameras are kept and resized, and the projection matrices are rebuilt for the new image size.

where λ is the depth of the point. NAVSIM provides the transform from each camera to the LiDAR frame, $(R'_v, \mathbf{t}'_v)$; its inverse gives $R_v = R_v'^\top$ and $\mathbf{t}_v = -R_v'^\top \mathbf{t}'_v$.

NAVSIM records eight cameras, and nuScenes six. We keep the six NAVSIM cameras whose positions match the nuScenes cameras (front, front left, front right, back left, back right, back) and drop the two additional side cameras. Each image is resized from $(W_0, H_0) = (1920, 1080)$ to the input size of UniDriveVLA, $(W, H) = (960, 544)$. The intrinsics are scaled accordingly,

$$K'_v = S K_v, \quad S = \text{diag}\left(\frac{W}{W_0}, \frac{H}{H_0}, 1\right), \quad (3.2)$$

and the model receives $P'_v = K'_v[R_v \mid \mathbf{t}_v]$. Figure 3.2 shows these steps.

3.3 UniDriveVLA experts

Each of UniDriveVLA’s three experts [1] processes its own set of tokens, the vectors a transformer operates on.

Understanding expert. The understanding expert is the pretrained vision–language model Qwen3-VL-2B [26]. It encodes the camera images and a text prompt into tokens F_u .

Perception expert. The perception expert follows SparseDrive [11]. It holds a fixed number of learned queries of two kinds: object queries, which detect other road users, and map queries, which detect map elements such as lane boundaries. Each query is updated with image features read through the projections P_v' of Section 3.2. Its output tokens are the perception tokens $\phi_s \in \mathbb{R}^{T_s \times 256}$ of scene s .

Action expert. The action expert generates the trajectory (Section 3.4). It processes two kinds of tokens. Two status tokens describe the ego vehicle: one encodes the ego history $\mathbf{h}$, the other encodes the command c together with the ego vehicle’s current motion state (10 values). During training the motion state is taken from the recorded data. At inference the perception expert estimates it, as in the released model; the ego state recorded by NAVSIM is not used. Six action tokens $\alpha_1, \dots, \alpha_T$ represent the trajectory being generated, one token per future position.

Masked joint attention. Each expert is a stack of transformer layers, and the three stacks have the same number of layers. The experts do not only exchange their final outputs. In every layer, the tokens of all three experts take part in one shared attention operation, while each expert keeps its own weights. In attention, every token computes three vectors from its current representation: a query, which describes what the token looks for; a key, which describes what the token contains; and a value, the information the token passes on. For the tokens F_g of expert $g \in \{u, p, a\}$ (understanding, perception, action) in one layer,

$$Q_g = F_g W_g^Q, \quad K_g = F_g W_g^K, \quad V_g = F_g W_g^V, \quad (3.3)$$

where the weight matrices W_g^Q , W_g^K , and W_g^V belong to expert g . The queries of the three experts are stacked into one matrix $Q = [Q_u; Q_p; Q_a]$, likewise for K and V . The attention output is

$$Z = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad (3.4)$$

where d_k is the length of a key. Each row of Z is a weighted sum of the values of all tokens, with weights given by how well the token’s query matches the other tokens’ keys. The mask M decides which tokens a token may attend to. Its entry is 0 if the token of the row (the query) may attend to the token of the column (the key), and $-\infty$ otherwise, which sets the corresponding weight to zero. Understanding tokens attend only to earlier understanding tokens. Perception tokens attend to understanding and perception tokens. Action tokens attend to the tokens of all three experts (Figure 3.3). Information therefore flows from the understanding and perception experts into the action expert, while the action expert does not influence the other two. The rows of Z that belong to expert g are that expert’s attention output in this layer. The expert processes them further with its own feed-forward layer and passes the result to its next layer.

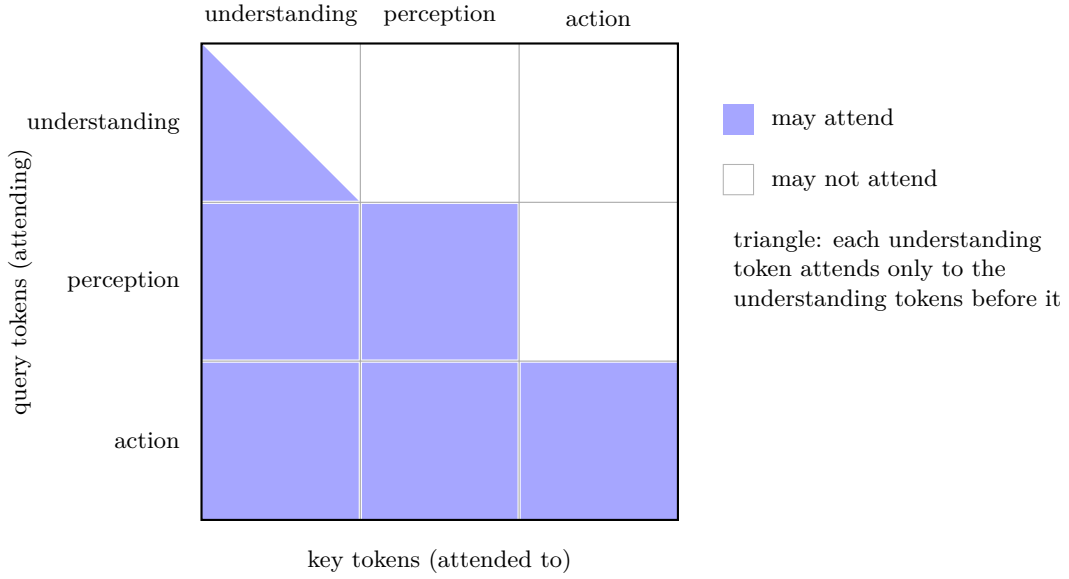


Figure 3.3: **Attention mask of masked joint attention**, applied in every layer. Each row shows which tokens the queries of one expert may attend to. The action expert attends to all tokens; no expert attends to the action tokens.

3.4 Trajectory generation with flow matching

The action expert generates trajectories with flow matching [15]. Let $\mathbf{a} \in \mathbb{R}^{T \times 2}$ be the recorded trajectory, written as displacements between consecutive positions and standardised with the mean and standard deviation of the training set. For noise $\epsilon \sim \mathcal{N}(0, I)$ and time $t \sim \mathcal{U}(0, 1)$, flow matching defines the point

$$\mathbf{x}_t = (1 - t) \mathbf{a} + t \epsilon \quad (3.5)$$

on a straight path from $\mathbf{a}$ ($t = 0$) to ϵ ($t = 1$), whose velocity is $\mathbf{u} = \epsilon - \mathbf{a}$. Each action token is computed from the corresponding row of $\mathbf{x}_t$ and an embedding of t . The action expert's output layer maps the action tokens to a predicted velocity $v_\theta(\mathbf{x}_t, t, s)$. Here s stands for the scene: the understanding and perception tokens that the action tokens attend to.

The training loss has two terms. The first compares the predicted velocity with $\mathbf{u}$. The second compares positions in metres. From the predicted velocity $v = v_\theta(\mathbf{x}_t, t, s)$, the recorded trajectory is estimated as $\hat{\mathbf{a}} = \mathbf{x}_t - t v$. Let $P(\cdot)$ convert standardised displacements back to metres and add them up into positions. The loss for one sample is

$$\ell(v) = w(t) \|v - \mathbf{u}\|^2 + \beta \|P(\hat{\mathbf{a}}) - P(\mathbf{a})\|^2, \quad \beta = 0.5, \quad (3.6)$$

and $\mathcal{L}_{\text{fm}}$ is its mean over the training samples. In the second term, the gradient of each position passes only through the last three displacements added to it. This prevents large gradients through the cumulative sum. The weight

$$w(t) = \frac{\min\{\text{SNR}(t), \gamma\}}{\text{SNR}(t)}, \quad \text{SNR}(t) = \left(\frac{1-t}{t}\right)^2, \quad \gamma = 5, \quad (3.7)$$

reduces the first term for small t , where the task is easiest [27].

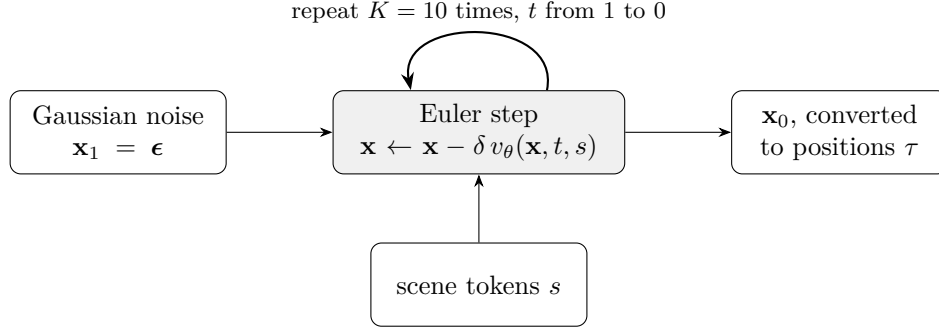


Figure 3.4: **Trajectory generation with flow matching.** Starting from Gaussian noise, the action expert applies $K = 10$ Euler steps with the predicted velocity v_θ to obtain a trajectory.

To generate a trajectory, the action expert starts from $\mathbf{x}_1 = \epsilon$ and takes $K = 10$ Euler steps of size $\delta = 1/K$ towards $t = 0$,

$$\mathbf{x}_{t-\delta} = \mathbf{x}_t - \delta v_\theta(\mathbf{x}_t, t, s). \quad (3.8)$$

The result $\mathbf{x}_0$ is converted back to metres and accumulated into the positions τ . The only random input is ϵ . Trained on one recorded trajectory per scene, the released action expert returns nearly the same trajectory for different ϵ (Section 4.4). Figure 3.4 shows the generation.

3.5 Mode embeddings

To obtain N different candidates, we add a table of N learned vectors $\mathbf{M} \in \mathbb{R}^{N \times d}$, where $d = 1024$ is the width of the action expert. Mode n adds row $\mathbf{M}_n$ to every action token before the first layer,

$$\alpha_k^{(n)} = \alpha_k + \mathbf{M}_n, \quad k = 1, \dots, T, \quad n = 0, \dots, N - 1. \quad (3.9)$$

The predicted velocity then depends on the mode, $v_\theta(\mathbf{x}_t, t, s, n)$, and the same scene and noise give a different trajectory $\tau^{(n)}$ for each mode. This plays the role of DrivoR’s learned trajectory queries [2]. We use $N = 8$; the table has $N \cdot d = 8192$ parameters. Mode $n = 0$ is the *default mode*, the candidate the planner returns without a scorer.

3.6 Relaxed winner-takes-all training

If all modes were trained towards the same recorded trajectory, they would become identical. Winner-takes-all training [17] avoids this by updating, for each scene, only the mode with the lowest loss. We use its relaxed form [28]: the mode with the lowest loss receives most of the update, and the other modes receive a small share (Figure 3.5). The loss is the flow-matching loss of Equation (3.6). It measures how far the velocity and the positions predicted by a mode are from those of the recorded trajectory, so the winner is the mode that best reproduces the human driver’s trajectory. No driving score is used in this training; EPDMS is used only by the scorer (Section 3.9). For a batch of B scenes, we draw one (ϵ_i, t_i) per scene i , shared by all modes, and compute the loss of each mode with Equation (3.6),

$$\ell_{i,n} = \ell(v_\theta(\mathbf{x}_{t_i}, t_i, s_i, n)). \quad (3.10)$$

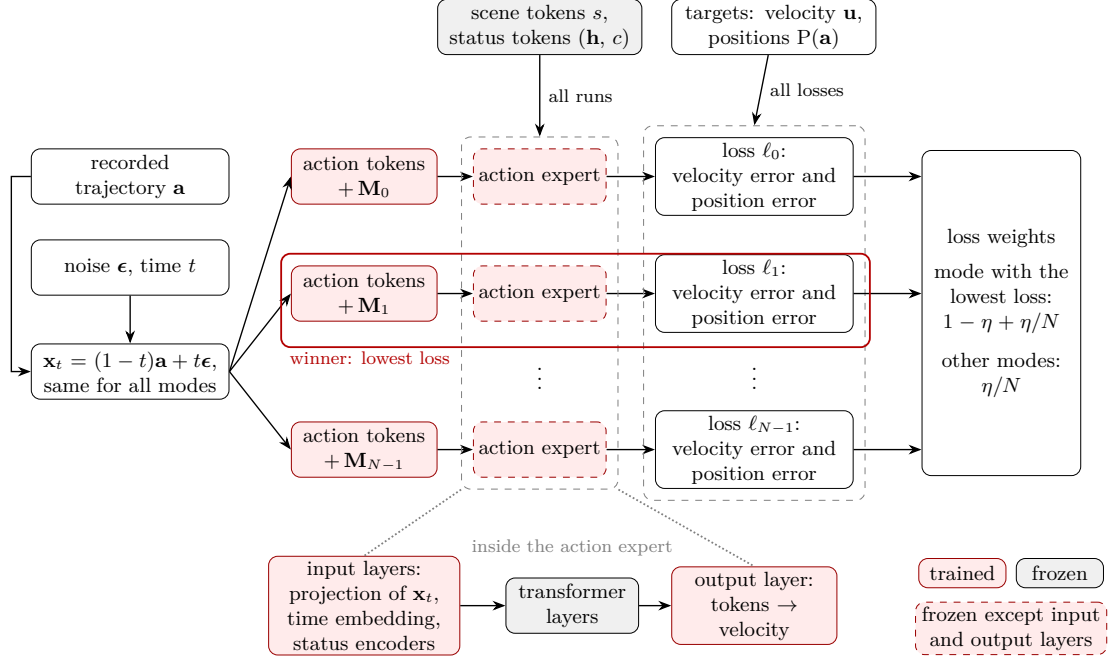


Figure 3.5: **Training of the mode embeddings.** For one recorded scene, the same noisy trajectory $\mathbf{x}_t$ is passed through the action expert once per mode, each time with a different mode embedding $\mathbf{M}_n$ added to the action tokens. Each run predicts a velocity, and its loss ℓ_n combines the error of this velocity and the error of the resulting positions (Equation (3.6)). The mode with the lowest loss receives most of the update; the other modes receive a small share, set by $\eta = 0.05$.

The training loss is

$$\mathcal{L}_{\text{wta}} = \frac{1}{B} \sum_{i=1}^B \left[(1 - \eta) \min_n \ell_{i,n} + \frac{\eta}{N} \sum_{n=0}^{N-1} \ell_{i,n} \right], \quad (3.11)$$

with $\eta = 0.05$. The first term updates only the winning mode, since the derivative of $\min_n \ell_{i,n}$ with respect to $\ell_{i,m}$ is 1 for the winning mode and 0 for the others. The modes thus specialise on different scenes. The second term gives every mode a small gradient, so that a mode that never wins is still trained.

The trained parameters are

- the mode table $\mathbf{M}$;
- the input layers of the action expert: the projection of $\mathbf{x}_t$ into the action tokens, the embedding of the time t , and the encoders that compute the status tokens from the ego history and the command;
- the output layer of the action expert, which maps the action tokens to the velocity.

Together these have about 5.3 million parameters. All other parameters of UniDrive-VLA are frozen (Table 3.1). Training uses the nuScenes training split; Table 4.1 lists the settings.

Table 3.1: Trained and frozen components.

Component	Training data	Status	Parameters
Understanding expert (Qwen3-VL-2B), perception expert, and transformer layers of the action expert	images and text, nuScenes	frozen	≈ 2.7 B
Action expert, input and output layers, and mode table	nuScenes	trained	≈ 5.3 M
Learned scorer	navhard, five-fold cross-validation	trained	≈ 0.13 M

3.7 Conversion to NAVSIM

UniDriveVLA outputs six positions over 3 s, whereas NAVSIM expects eight poses (x, y, θ) over 4 s. The conversion has three steps (Figure 3.6).

Axes. UniDriveVLA stores each position as (lateral, forward), (ℓ_k, f_k) . NAVSIM uses x forward and y left, so

$$x_k = f_k, \quad y_k = \zeta \ell_k, \quad \zeta = -1. \quad (3.12)$$

We verified the sign ζ on navhard: converting NAVSIM’s recorded future positions into the model’s convention and back reproduces them only with $\zeta = -1$. The wrong sign would mirror every turn.

Horizon. UniDriveVLA predicts $T = 6$ positions (3 s), NAVSIM expects $T' = 8$ (4 s). The two missing positions continue the trajectory at the speed and direction of its last step: each adds the last displacement $\mathbf{p}_T - \mathbf{p}_{T-1}$ once more,

$$\mathbf{p}_k = \mathbf{p}_T + (k - T)(\mathbf{p}_T - \mathbf{p}_{T-1}), \quad k = 7, 8. \quad (3.13)$$

This extension only makes the trajectory compatible with NAVSIM v2; in a turn it continues straight. Section 5.3 discusses its possible effect.

Heading. Each pose receives the direction of the displacement that leads to it,

$$\theta_k = \text{atan2}(y_k - y_{k-1}, x_k - x_{k-1}), \quad k = 1, \dots, T', \quad \mathbf{p}_0 = (0, 0). \quad (3.14)$$

3.8 EPDMS

The scorer predicts the terms of EPDMS, the metric of NAVSIM v2 [5], so we state its exact form. For a trajectory τ in a scene s , NAVSIM v2 computes nine terms in $[0, 1]$. Four are penalty terms:

- no at-fault collision (NC): the ego vehicle does not cause a collision;
- drivable-area compliance (DAC): the ego vehicle stays on the drivable area;
- driving-direction compliance (DDC): the ego vehicle does not drive against the direction of its lane outside intersections;

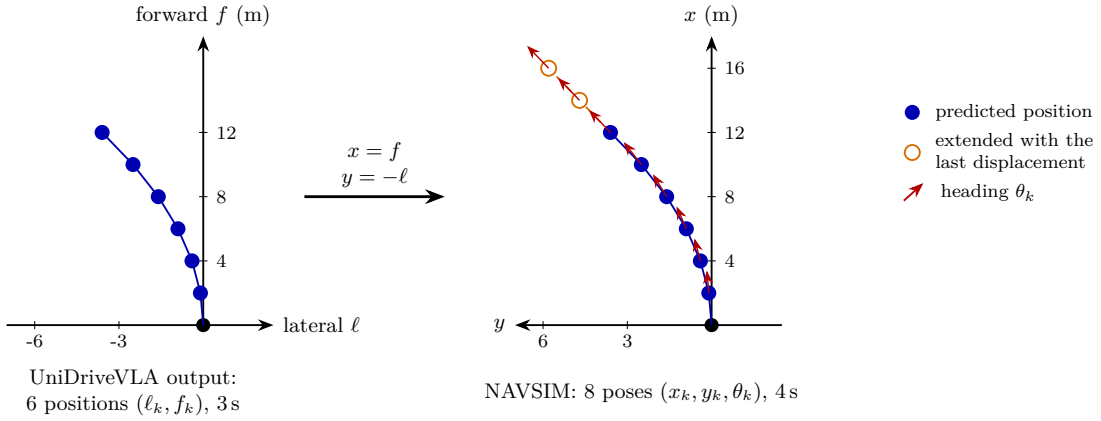


Figure 3.6: **Conversion to NAVSIM**, shown for a left turn. The model measures the lateral coordinate to the right, NAVSIM measures y to the left; both panels show the same path. The six predicted positions are extended to eight by repeating the last displacement, and each pose receives the heading of the displacement that leads to it.

- traffic-light compliance (TLC): the ego vehicle does not pass a red light.

Five are quality terms:

- ego progress (EP): how far the ego vehicle advances along its route;
- time to collision (TTC): the ego vehicle keeps a safe time gap to other road users;
- history comfort (HC): acceleration and jerk stay within comfort limits when the trajectory is joined to the vehicle's recorded motion before it;
- lane keeping (LK): the ego vehicle stays close to the centre of its lane;
- extended comfort (EC): the trajectory is consistent with the one planned in the previous frame, without jumps in acceleration or jerk.

NAVSIM v2 combines the nine terms of one trajectory in one scene into a single score. We call this score the *stage score*, because it is computed separately for every scene of both stages before EPDMS combines them:

$$m(\tau, s) = \left(\prod_{c \in \mathcal{P}} c \right) \cdot \frac{\sum_{j \in \mathcal{Q}} w_j q_j}{\sum_{j \in \mathcal{Q}} w_j}, \quad (3.15)$$

where $\mathcal{P}$ is the set of penalty terms, $\mathcal{Q}$ the set of quality terms, and the weights are $(w_{\text{EP}}, w_{\text{TTC}}, w_{\text{HC}}, w_{\text{LK}}, w_{\text{EC}}) = (5, 5, 2, 2, 2)$. If any penalty term is zero, the stage score is zero.

NAVSIM v2 evaluates each scene s in two stages [5] (Figure 3.7). In the first stage, the planner predicts a trajectory τ from the recorded scene. This trajectory receives the stage score $m(\tau, s)$ and ends at the position $\hat{x}$.

The second stage tests the planner four seconds later, from positions near the ones it may reach. NAVSIM renders J new scenes s_j in which the ego vehicle starts at positions x_j around the recorded position of the human driver after four seconds. The planner predicts a new trajectory τ_j in each of them, which receives the stage score $m(\tau_j, s_j)$. The planner's own first-stage trajectory ends at $\hat{x}$, not necessarily at the

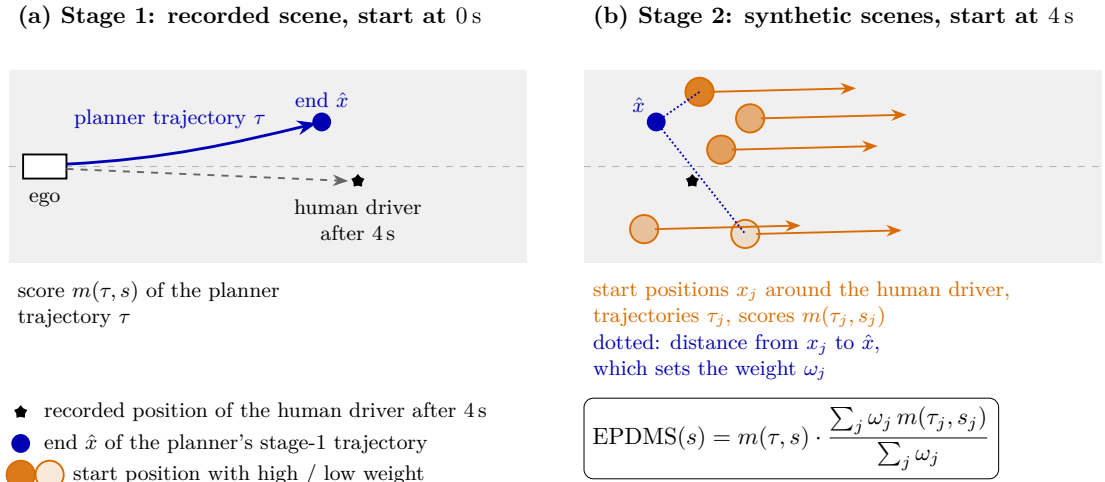


Figure 3.7: **Two-stage scoring in NAVSIM v2.** (a) First stage from the recorded scene. (b) Second stage from synthetic start positions, weighted by their distance to $\hat{x}$ (Section 3.8).

human driver's position, so each stage score of the second stage is weighted by how close its start position x_j is to $\hat{x}$.

EPDMS multiplies the first-stage score with the weighted mean of the second-stage scores:

$$\text{EPDMS}(s) = m(\tau, s) \cdot \frac{\sum_{j=1}^J \omega_j m(\tau_j, s_j)}{\sum_{j=1}^J \omega_j}, \quad \omega_j = \exp\left(-\frac{\|x_j - \hat{x}\|^2}{2\sigma^2}\right), \quad (3.16)$$

where σ is a fixed constant. The reported EPDMS is the mean of $\text{EPDMS}(s)$ over all scenes, multiplied by 100.

3.9 Learned scorer

The scorer g_ϕ predicts the nine EPDMS terms of a candidate from the candidate and the perception tokens (Figure 3.8). It never uses the recorded future. An MLP encodes the candidate's six positions into a query, and a linear layer projects the perception tokens into keys and values:

$$\mathbf{q}^{(n)} = \text{MLP}_c(\text{vec}(\tau^{(n)})) \in \mathbb{R}^{128}, \quad \Phi_s = \phi_s W_p + \mathbf{b}_p \in \mathbb{R}^{T_s \times 128}. \quad (3.17)$$

Multi-head cross-attention with four heads combines them, and a second MLP with a sigmoid outputs the predicted terms,

$$\mathbf{z}^{(n)} = \text{MHA}(\mathbf{q}^{(n)}, \Phi_s, \Phi_s), \quad \hat{\mathbf{g}}^{(n)} = \sigma\left(\text{MLP}_h(\text{LN}(\mathbf{z}^{(n)}))\right) \in [0, 1]^9, \quad (3.18)$$

where LN is layer normalisation. The predicted score $\hat{s}^{(n)}$ is obtained by inserting $\hat{\mathbf{g}}^{(n)}$ into Equation (3.15), and the scorer selects

$$\hat{n} = \arg \max_n \hat{s}^{(n)}. \quad (3.19)$$

Because the terms are predicted separately, the weights w_j in Equation (3.15) can be changed after training, as in DrivoR [2]. The scorer has about 0.13 million parameters.

Training. The targets are the true terms $\mathbf{g}^{(n)}$ of each candidate, computed by NAVSIM. The loss for a batch of scenes is

$$\mathcal{L}_{\text{sc}} = \mathcal{L}_{\text{BCE}} + \lambda \mathcal{L}_{\text{rank}}, \quad \mathcal{L}_{\text{rank}} = \frac{1}{|S'|} \sum_{s \in S'} \frac{1}{N-1} \sum_{n \neq n_s^*} \max(0, \mu - (\hat{s}_s^{(n_s^*)} - \hat{s}_j^{(n)})), \quad (3.20)$$

where $\mathcal{L}_{\text{BCE}}$ is the binary cross-entropy between the predicted and the true terms, averaged over the nine terms and the C candidates of the batch,

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{9C} \sum_{n=1}^C \sum_{j=1}^9 \left[g_j^{(n)} \log \hat{g}_j^{(n)} + (1 - g_j^{(n)}) \log (1 - \hat{g}_j^{(n)}) \right]. \quad (3.21)$$

n_s^* is the candidate with the highest true score in scene s , and S' contains the scenes of the batch in which the candidates' true scores differ. The ranking term asks the best candidate's predicted score to exceed every other candidate's by the margin $\mu = 0.02$; we use $\lambda = 1$. The training settings and data are described in Section 4.1 and Section 4.7.

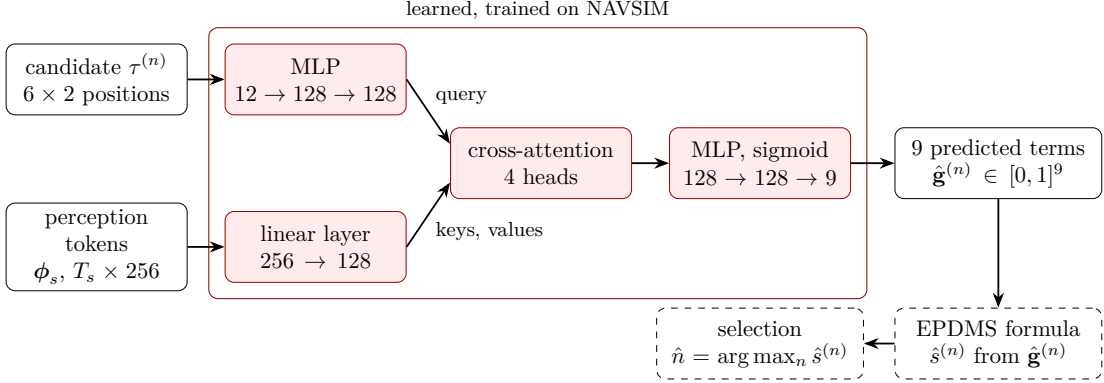


Figure 3.8: **The learned scorer.** For each candidate, an MLP encodes its six positions into a query that attends over the perception tokens. An MLP predicts the nine EPDMS terms from the result. The EPDMS formula combines them into one score, and the candidate with the highest score is selected. Only the parts inside the frame are learned.

3.10 Inference

Algorithm 1 lists the steps for one scene. The understanding and perception experts run once; their keys and values are cached and reused by every run of the action expert. All modes start from the same noise, so their candidates differ only through the mode embeddings.

Algorithm 1 Inference for one scene

Require: camera images and projections P'_v , ego history $\mathbf{h}$, command c

- 1: run the understanding and perception experts; cache their keys and values; keep ϕ_s
 - 2: draw $\epsilon \sim \mathcal{N}(0, I)$, shared by all modes
 - 3: **for** $n = 0, \dots, N - 1$ **do**
 - 4: $\mathbf{x}_1 \leftarrow \epsilon$
 - 5: **for** $t = 1, 1 - \delta, \dots, \delta$ **do**
 - 6: $\mathbf{x}_{t-\delta} \leftarrow \mathbf{x}_t - \delta v_\theta(\mathbf{x}_t, t, s, n)$
 - 7: **end for**
 - 8: $\tau^{(n)} \leftarrow$ positions from $\mathbf{x}_0$
 - 9: $\hat{s}^{(n)} \leftarrow \text{EPDMS}(g_\phi(\tau^{(n)}, \phi_s))$
 - 10: **end for**
 - 11: $\hat{n} \leftarrow \arg \max_n \hat{s}^{(n)}$
 - 12: **return** $\tau^{(\hat{n})}$ converted to NAVSIM poses (Section 3.7)
-

Chapter 4

Experiments

This chapter describes the experiments in the order in which they were run:

1. a check of the evaluation pipeline with the released UniDriveVLA (Section 4.3);
2. a test of whether the released model produces different trajectories for different random starting values (Section 4.4);
3. the training of the eight mode embeddings and the choice of the checkpoint (Section 4.5);
4. the evaluation of the eight candidates on navhard (Section 4.6);
5. the training and evaluation of the learned scorer (Section 4.7);
6. an ablation of the scorer (Section 4.8).

Sections 4.1 and 4.2 first describe the setup and the metrics.

4.1 Experimental setup

Data. The mode embeddings are trained on the nuScenes training split (28 130 samples), and the checkpoint is selected on the nuScenes validation split (Section 4.5). All driving evaluations use NAVSIM v2. Its warm-up split (220 scenes) is used to check the pipeline. All results are reported on navhard, the test split of NAVSIM v2. Navhard contains 450 real scenes and 5462 synthetic second-stage scenes generated from them, 5912 scenes in total. UniDriveVLA is not trained on NAVSIM v2 data.

Hardware. The mode embeddings were trained on eight NVIDIA A100 GPUs with 80 GB each. Each GPU processes one sample per iteration, so one iteration covers eight samples. In every iteration, the joint transformer of all three experts runs nine times: once without a mode embedding and once per mode. An iteration took about one minute, measured in a first run with the same setup. NAVSIM v2’s scoring runs on CPUs; scoring one candidate on all navhard scenes takes one to two hours.

Code. The code, the trained mode embeddings and the instructions to reproduce all results are available at <https://github.com/abdo75/unidrivevla-8-mode>.

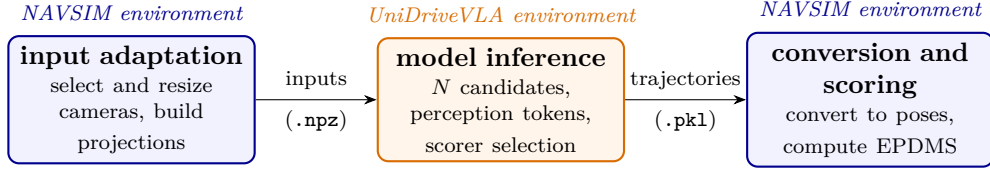


Figure 4.1: **Evaluation pipeline.** UniDriveVLA and NAVSIM run in separate software environments and only exchange files.

Settings. Table 4.1 lists the training settings of the mode embeddings and of the scorer. The planner outputs $N = 8$ candidates per scene.

Table 4.1: Training settings of the mode embeddings and the scorer.

Setting	Value
Number of modes N	8
Mode-embedding dimension d	1024
Mode initialisation	Gaussian, mean 0, standard deviation 0.08
Relaxed winner-takes-all share η	0.05
Optimiser	AdamW
Learning rate	1×10^{-4} ; mode table 5×10^{-4}
Schedule	cosine over 17 580 iterations, 500 warm-up iterations
Training data	nuScenes training split (28 130 samples)
Batch	8 GPUs $\times$ 1 sample, gradient accumulation over 4 iterations
Selected checkpoint	iteration 7000 (Section 4.5)
Scorer: optimiser	AdamW, learning rate 10^{-3}
Scorer: training	200 epochs per cross-validation group
Scorer: loss	BCE + ranking term, $\lambda = 1$, margin $\mu = 0.02$

Evaluation pipeline. UniDriveVLA and NAVSIM’s scoring code require incompatible versions of PyTorch, so they cannot run in one Python environment. The evaluation therefore runs in three steps that only exchange files (Figure 4.1). The NAVSIM side writes the model inputs (Section 3.2). The UniDriveVLA side writes the candidates and the scorer’s selection. The NAVSIM side converts the trajectories (Section 3.7) and scores them.

4.2 Evaluation metrics

EPDMS. The main metric is EPDMS (Section 3.8), multiplied by 100. All published NAVSIM v2 results use it.

Stage score. EPDMS evaluates a planner as a whole. It is not suited to analysing the selection among candidates, for two reasons. First, the stages are coupled: the trajectory chosen in the first stage decides where it ends, and therefore how much each second-stage scene counts (Section 3.8). Choosing a different candidate in one scene thus changes the score of other scenes, and an oracle that picks the best candidate

scene by scene is not defined. Second, every EPDMS value requires a separate scoring run of NAVSIM v2 (Section 4.1); we ran it for mode 0, for each single mode, and for the scorer.

To analyse the selection, we therefore also use the stage score $m(\tau, s)$ of Equation (3.15): the score of one trajectory τ in one scene s , combined from its nine terms, before the two stages are combined into EPDMS. It is computed for each of the 5912 navhard scenes, first-stage and second-stage, from that scene’s trajectory alone, so choosing a candidate in one scene affects only that scene. It is also the quantity the scorer predicts (Section 3.9). We report its mean over all scenes, multiplied by 100. Because 5462 of the 5912 scenes are second-stage scenes, this mean is dominated by the second stage. The stage score is used only to compare selection policies on our own candidates. Its values differ from EPDMS, and all comparisons with published planners use EPDMS.

For the candidates $\tau_s^{(n)}$, $n = 0, \dots, N - 1$, of a scene s in the set S of navhard scenes, we compare five selection policies:

$$\text{default} = \frac{1}{|S|} \sum_s m(\tau_s^{(0)}, s), \quad (4.1)$$

$$\text{random} = \frac{1}{|S|} \sum_s \frac{1}{N} \sum_n m(\tau_s^{(n)}, s), \quad (4.2)$$

$$\text{scorer} = \frac{1}{|S|} \sum_s m(\tau_s^{(\hat{n}_s)}, s), \quad (4.3)$$

$$\text{best-fixed} = \max_n \frac{1}{|S|} \sum_s m(\tau_s^{(n)}, s), \quad (4.4)$$

$$\text{oracle} = \frac{1}{|S|} \sum_s \max_n m(\tau_s^{(n)}, s). \quad (4.5)$$

The default always uses mode 0. Random is the expected score when a candidate is chosen at random in each scene; a scorer that uses information about the scene must exceed it. The scorer uses the candidate $\hat{n}_s$ selected by the learned scorer. The best fixed mode uses the same mode in every scene, chosen with the true scores of the evaluation scenes. The oracle uses the true score of every candidate, so it is an upper bound for any selector.

4.3 Pipeline check

We first scored a constant-velocity agent, which ignores the cameras and keeps the current speed, and then the released UniDriveVLA, on the warm-up split (Table 4.2). A wrong input conversion would reduce UniDriveVLA’s score to the level of the constant-velocity agent. UniDriveVLA scores 10.4 points higher. This check used the lateral sign $\zeta = +1$, which was corrected later (Section 3.7); it was not repeated with the corrected sign.

Table 4.2: Pipeline check on the NAVSIM v2 warm-up split (220 scenes).

Agent	EPDMS
Constant velocity (ignores the cameras)	18.5
UniDriveVLA (released, single trajectory)	28.9

4.4 Diversity of the released model

On five warm-up scenes, we ran the released action expert six times per scene with different random starting values. The endpoints of the six trajectories differed by about 0.1 m at a distance of about 26 m. The released model therefore does not provide candidates by itself, which is the reason for the mode embeddings of Section 3.5.

4.5 Training the mode embeddings

We trained the mode embeddings for 17 580 iterations. Since each iteration covers eight samples, this corresponds to five epochs of the nuScenes training split. To choose a checkpoint without using NAVSIM data, we evaluated checkpoints on 500 samples of the nuScenes validation split. For 72 of them the recorded future is shorter than 3 s, so 428 samples remain. On these, we measure the minimum average displacement error over the N candidates,

$$\text{minADE}_N = \min_n \frac{1}{T} \sum_{k=1}^T \|\mathbf{p}_k^{(n)} - \mathbf{p}_k^*\|, \quad (4.6)$$

with recorded positions $\mathbf{p}_k^*$. It measures whether *some* candidate is close to the recorded trajectory. Figure 4.2 shows that minADE_8 falls from 0.98 m after 1000 iterations to 0.80 m after 7000 and does not improve afterwards (0.82 m after 17 580). We therefore use the checkpoint at iteration 7000, which corresponds to two epochs and about five days of training. The error of mode 0 alone stays between 1.16 m and 1.28 m.

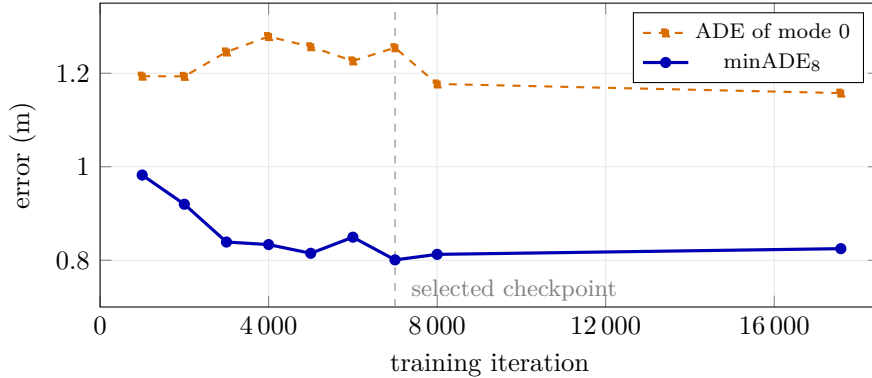


Figure 4.2: **Checkpoint selection for the mode training** on 428 nuScenes validation samples. Checkpoints were evaluated every 1000 iterations up to 8000 and at the end of training (17 580).

4.6 Candidates on navhard

We then scored each of the eight candidates on all navhard scenes, using the same mode in every scene (Table 4.3). Mode 0 scores 10.0 EPDMS and mode 6, the best single mode, 12.0. In mean stage score, mode 0 reaches 24.4, a random candidate 24.2, and the oracle 34.6, 10.3 points (42 %) above mode 0. In 53.1 % of the scenes all eight candidates receive the same stage score. Mode 0 scores exactly zero in 63.2 % of the scenes.

Table 4.3: Scores of each of the eight candidates on navhard when the same mode is used in every scene.

Mode	0	1	2	3	4	5	6	7
EPDMS	10.0	7.9	9.2	9.3	9.8	10.5	12.0	8.7
Mean stage score	24.4	21.2	24.4	23.7	24.5	24.8	27.0	23.6

4.7 Learned scorer

Training data. DrivoR trains its scorer on NAVSIM’s training split (navtrain) and evaluates it on navhard. We could not do the same within our compute budget. The scorer’s training labels are the true EPDMS terms of each candidate. Producing them for navtrain would require scoring all eight candidates of every navtrain scene, and navtrain has many times more scenes than navhard. For navhard, these labels already exist, because computing them is part of the evaluation in Section 4.6.

A scorer must not be evaluated on the scenes it was trained on. We therefore use five-fold cross-validation (Figure 4.3a). We split the 5912 navhard scenes at random into five groups of about 1180 scenes and train five scorers. Each scorer is trained on four groups and selects the candidates in the remaining group. In this way, every scene is selected by a scorer that was not trained on it.

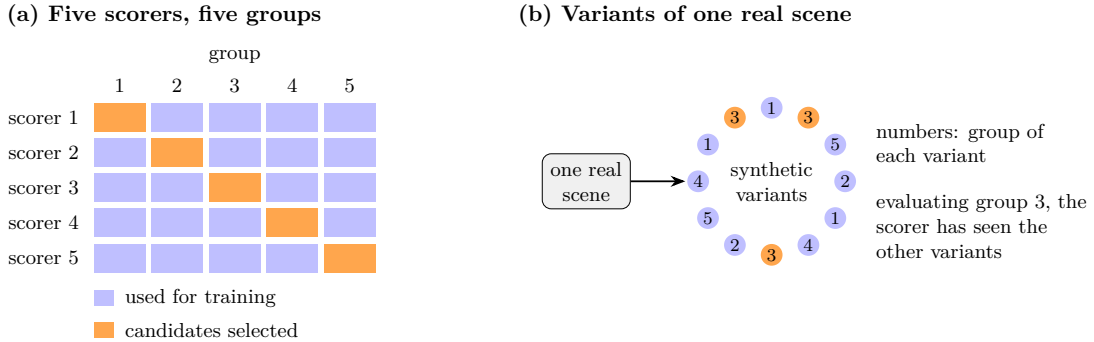


Figure 4.3: **Cross-validation of the scorer on navhard.** (a) Each of five scorers is trained on four groups and selects the candidates in the remaining group. (b) The groups are formed at random over all scenes, so the synthetic variants of one real scene fall into different groups.

Limits of this evaluation. Each of the 450 real scenes of navhard has about twelve synthetic variants, which show the same road and traffic from slightly shifted positions of the ego vehicle. Because the groups are formed at random over all scenes, the variants of one real scene end up in different groups (Figure 4.3b). A scorer that selects a candidate in one variant may therefore have been trained on other variants of the same real scene. This makes its task easier than on a scene it has never seen, so the result may be higher than on new scenes. Forming the groups by real scene would avoid this. In addition, NAVSIM v2 does not allow training on navhard for leaderboard entries, so our result with the scorer cannot be submitted as one.

Results. Table 4.4 compares the five selection policies in mean stage score. The scorer reaches 26.8, against 24.4 for mode 0 and 24.2 for a random candidate. It recovers 24 %

of the distance from mode 0 to the oracle. It is 0.2 points below the best fixed mode, which is chosen with the true scores. Per scene, the scorer selects a better candidate than mode 0 in 29.0 % of the scenes, an equally scored one in 61.8 %, and a worse one in 9.3 %. Figure 4.4 shows how often it selects each mode: mode 6, the best single mode, in 46.5 % of the scenes, and mode 1, the weakest single mode in EPDMS (7.9), in 20.1 %.

Table 4.4: Selection policies on navhard, mean stage score.

Policy (over 8 candidates)	mean stage score
Default (mode 0)	24.4
Random candidate	24.2
Learned scorer	26.8
Best fixed mode in hindsight (mode 6)	27.0
Oracle (best of 8 per scene)	34.6

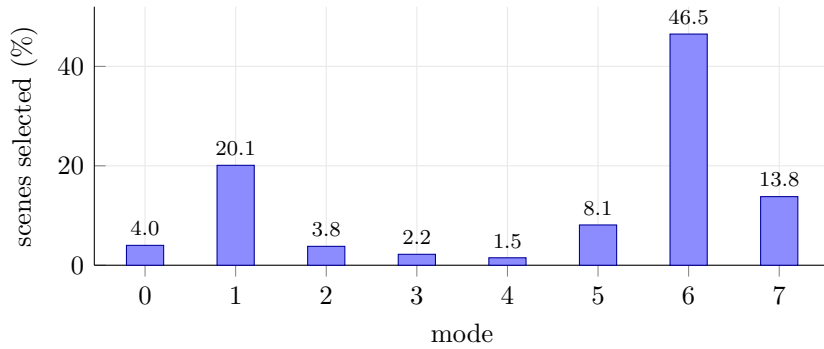


Figure 4.4: **Modes selected by the learned scorer** in the 5912 navhard scenes.

Comparison with published planners. Table 4.5 compares the planner with published planners in EPDMS. To our knowledge, UniDriveVLA has not been evaluated on NAVSIM before; its authors report results on nuScenes and in closed-loop simulation. To our knowledge, SparseDrive and VAD have no published navhard results, so they are not listed. Our pipeline scores the constant-velocity agent at 11.5, close to the published 10.9. The released UniDriveVLA was not scored on navhard with the corrected lateral sign, so the table contains no result of the released model; mode 0 of the eight-mode model is the single-trajectory reference. With the scorer, the planner scores 12.7, 27 % more than mode 0 (10.0) and 0.7 points more than the best fixed mode (12.0). It equals the ego-status MLP (12.7), scores above the constant-velocity agent, and scores below Latent TransFuser (23.1), the candidate-selection planners GTRS (45.0), ZTRS (48.1), and DrivoR (48.3), and PDM-Closed (51.3).

4.8 Ablation of the scorer

Finally, we reduced the influence of the scorer step by step. Mode 0 is replaced by the scorer’s choice only when the predicted score of that choice exceeds the predicted score of mode 0 by more than a margin τ (in points). With $\tau = \infty$, the scorer is removed and mode 0 is always used. Table 4.6 shows that the mean stage score decreases from 26.8 at $\tau = 0$ to 24.4 at $\tau = \infty$. This ablation uses the evaluation scenes, so it was not

Table 4.5: EPDMS (higher is better) on navhard. The reference planners are trained on NAVSIM; values from [5] and from Table 3 of [2]. UniDriveVLA is not trained on NAVSIM; the learned scorer is trained on navhard with five-fold cross-validation.

Method	EPDMS $\uparrow$
Constant velocity [5]	10.9
Ego-status MLP [5]	12.7
Latent TransFuser [12, 5]	23.1
GTRS-D [19, 2]	45.0
ZTRS [20, 2]	48.1
DrivoR [2]	48.3
PDM-Closed [9]	51.3
UniDriveVLA, eight modes, mode 0	10.0
UniDriveVLA, eight modes, best fixed mode in hindsight	12.0
UniDriveVLA, eight modes + learned scorer	12.7

Table 4.6: Ablation of the scorer on navhard (mean stage score). Mode 0 is replaced only when the predicted gain exceeds τ ; $\tau = 0$ is the plain arg max, and $\tau = \infty$ removes the scorer.

Margin τ	0	1	2	5	10	∞ (always mode 0)
Mean stage score	26.8	26.2	26.2	26.1	25.6	24.4

used to choose τ ; the reported scorer uses $\tau = 0$, fixed in advance. The other ablations listed in Section 5.3 could not be run.

Chapter 5

Discussion

This chapter interprets the results of Chapter 4, shows examples of individual scenes, and discusses the limitations of this work and possible next steps.

5.1 Interpretation of the results

Why the scorer improves the planner. The eight candidates of a scene differ in ways that EPDMS rewards: an oracle that picks the best candidate in each scene reaches a mean stage score of 34.6, against 24.4 for mode 0. The scorer recovers about a quarter of this difference. It is above a random choice of candidate (24.2), so its predicted ranking carries information about which candidate scores higher. Its choice depends on the scene: it selects mode 6, the best single mode, in about half of the scenes, but it also selects mode 1, the weakest mode when used in every scene, in a fifth of the scenes. A fixed choice of mode cannot vary in this way. Compared with mode 0, the scorer’s choice is better three times as often as it is worse (29.0 % against 9.3 % of the scenes). This is consistent with the ablation in Section 4.8: every margin that limits how often the scorer may replace mode 0 lowers the result.

Why mode 0 is a weak default. Relaxed winner-takes-all training updates mainly the mode that is closest to the recorded trajectory in each scene. The modes therefore specialise on different scenes, and no mode, including mode 0, is trained to be good in every scene. The training curve shows this: minADE_8 improves while the error of mode 0 alone does not (Figure 4.2). The scorer selects mode 0 in only 4.0 % of the scenes.

Why the absolute score is low. Mode 0 scores exactly zero in 63.2 % of the navhard scenes. A zero requires at least one penalty term to be zero: an at-fault collision, leaving the drivable area, driving against the lane direction, or passing a red light. In 53.1 % of the scenes all eight candidates receive the same stage score, so no selector can improve these scenes. Most of the gap to the strongest planners therefore most likely comes from the candidates themselves, not from the selection. Table 5.1 compares the planners on the design choices that differ.

Three differences are the most likely reasons for the gap to DrivoR. First, UniDriveVLA is trained only on nuScenes, whereas navhard is recorded with other vehicles and cameras and partly in other cities; DrivoR is trained on navtrain. Second, UniDriveVLA’s

Table 5.1: Design of the compared planners. “Training data” is the data the planner is trained on; “Backbone” states how the image encoder is adapted to NAVSIM; “Cand.” is the number of candidate trajectories per scene. PDM-Closed reads the map and the positions of other road users from the dataset annotations; the learned planners use only sensors. Values for DrivoR, ZTRS, and GTRS-D from Table 3 of [2], the others from [5, 4].

Method	Inputs	Training data	Backbone	Cand.	EPDMS
PDM-Closed [9]	map, tracks	rule-based	—	many	51.3
DrivoR [2]	4 cameras	navtrain	ViT-S (LoRA)	64	48.3
ZTRS [20]	cameras	navtrain	V2-99 (trained)	many	48.1
GTRS-D [19]	cameras	navtrain	V2-99 (trained)	many	45.0
Latent TransFuser [12, 5]	cameras	navtrain	trained	1	23.1
Ego-status MLP [5]	ego only	navtrain	—	1	12.7
UniDriveVLA + scorer (ours)	6 cameras	nuScenes	frozen	8	12.7

backbone is frozen: we train about 5.3 million of its roughly 2.7 billion parameters (under 0.2%), so the perception and understanding experts cannot adapt to NAVSIM’s roads, whereas DrivoR adapts its image encoder. Third, our candidates often receive the same score. DrivoR’s own experiments show that more candidates help when the planner is trained on the target data, with the gain levelling off at about 64 candidates [2]. With a frozen backbone trained elsewhere, additional modes would likely receive the same score in the same scenes. DrivoR reaches a drivable-area compliance of 88.4 in the second stage [2], so most of its trajectories stay on the road. In the case studies of Section 5.2, mode 0 leaves the drivable area at turns. The lateral sign of the conversion was verified on navhard (Section 3.7). The most likely cause is the domain gap: the perception expert places the trajectories using road geometry learned on nuScenes. Two further factors may contribute. First, the model predicts 3 s, and the fourth second is added by repeating the last step (Section 3.7). In a turn, this extension continues straight and can leave the drivable area. Second, the second-stage scenes, which make up most of navhard, consist of synthetic camera images, which differ from the real images the model was trained on. Neither effect was measured separately. The mode training converged on nuScenes (Figure 4.2); whether more training would change the result on NAVSIM v2 was not tested.

PDM-Closed is a different case. It reads the map and the tracks of other road users from the dataset annotations and scores its candidates with the same rules that EPDMS is built on. A planner that uses only cameras is not expected to match it.

5.2 Case studies

Figure 5.1 shows three navhard scenes in which mode 0 leaves the drivable area (drivable-area compliance of zero) while another candidate stays on it. The scenes were chosen from the first-stage scenes by this criterion. In all three, the human driver turns at an intersection. The eight candidates lie close to each other, cover a shorter distance than the recorded trajectory, and turn less sharply. In each scene another candidate stays on the drivable area, so a selector can avoid leaving it, but none of

the candidates follows the turn of the human driver. Part of this may come from the straight extension of the fourth second (Section 3.7).

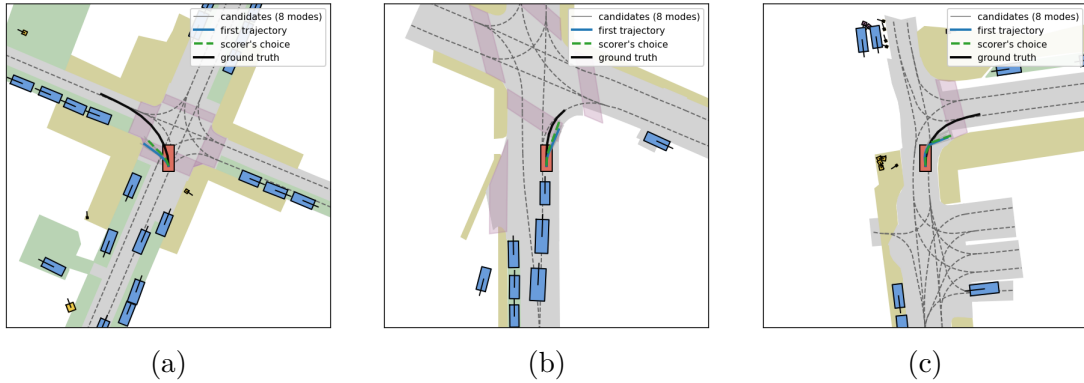


Figure 5.1: **Case studies on navhard.** Three scenes in which mode 0 leaves the drivable area while another candidate stays on it. Grey: the eight candidates; blue (“first trajectory”): mode 0; green, dashed: the scorer’s choice; black (“ground truth”): the recorded trajectory of the human driver; red box: ego vehicle. (a) Left turn at an intersection. (b) Right turn into a crossing road. (c) Right turn at a junction.

5.3 Limitations

- *Domain and frozen backbone.* UniDriveVLA is trained only on nuScenes and its backbone stays frozen, so the quality of the candidates on NAVSIM v2 is limited by how well the model transfers. The second-stage scenes add synthetic camera images.
- *Horizon extension.* The fourth second of every trajectory is a straight continuation of the last predicted step, added only for compatibility with NAVSIM v2. Its effect on EPDMS was not measured.
- *No navhard result for the released model.* The released UniDriveVLA was evaluated only on the warm-up split, with the uncorrected lateral sign. The comparison with it on navhard is therefore missing.
- *Scorer evaluation.* The cross-validation on navhard may be optimistic, and the result cannot be submitted to the leaderboard (Section 4.7).
- *Stage score and oracle.* With more compute, EPDMS could also be computed for the oracle’s and a random selection, by scoring each selection in a separate NAVSIM v2 run. We report these policies only in mean stage score, which is dominated by second-stage scenes and cannot be compared with published results.
- *Missing ablations.* The number of modes, the relaxed winner-takes-all share, and the mode initialisation were fixed, because each setting requires a new multi-day training run. For the same reason, the mode embeddings were not compared in EPDMS with candidates from different random starting values, which Section 4.4 shows to be almost identical. A per-term breakdown of EPDMS is not available, so the attribution of zero scores to specific penalty terms rests on the case studies.
- *Ego state.* At inference the model uses its own estimate of the ego vehicle’s motion state. Using the ego state recorded by NAVSIM v2 instead was not tested.

- *Single run.* The mode embeddings were trained once, and the cross-validation used one random split. The variance of the results is unknown.
- *Computational cost.* At inference, the action expert runs $N = 8$ times with $K = 10$ steps each, while the understanding and perception experts run once. During training, the joint transformer runs $N + 1 = 9$ times per iteration.
- *Evaluation protocol.* NAVSIM v2 scores each trajectory once and does not feed it back to the planner, so the planner is not tested in closed loop.

5.4 Future work

- *Training on NAVSIM.* Training the action expert and the mode embeddings, and in a second step the perception expert, on navtrain would address the domain gap and give the scorer candidates that stay on the road.
- *Scorer evaluation.* Training the scorer on navtrain, or forming the cross-validation groups by real scene, would remove the optimism of the current evaluation and allow a leaderboard submission.
- *A stronger default.* Adding the trajectory of the released planner as an additional candidate would give the scorer a default that is not specialised by winner-takes-all training.
- *Missing analyses.* EPDMS for the released model and for the oracle’s selection on navhard, a per-term breakdown of EPDMS, several training runs, and a comparison of different numbers of modes would complete the evaluation.
- *Full horizon.* Training the action expert to predict the full 4s of NAVSIM v2 would remove the straight extension of the last second.
- *Closed-loop evaluation.* Testing the planner in a closed-loop simulator would show whether the selection also helps when errors accumulate.

5.5 Conclusion

This thesis tested whether DrivoR’s approach of generating several candidate trajectories and selecting one with a learned scorer improves the vision–language–action model UniDriveVLA, and whether the combination reaches state-of-the-art results, without training UniDriveVLA on NAVSIM data. We built a pipeline that evaluates UniDriveVLA with EPDMS on NAVSIM v2. We added eight mode embeddings to the action expert and trained them with relaxed winner-takes-all, so that the model produces eight different candidates. A learned scorer predicts the EPDMS terms of each candidate and selects one. On navhard, the scorer raises EPDMS from 10.0 for mode 0 to 12.7, above every single mode. In our cross-validated evaluation, candidate selection therefore improves the default mode of a vision–language–action model trained on another dataset. It does not reach state-of-the-art results: the planner scores below planners trained on NAVSIM, such as DrivoR (48.3), most likely because many candidates violate a penalty term such as the drivable area. Adapting the model to the target data is the main remaining step.

Bibliography

- [1] Yongkang Li, Lijun Zhou, Sixu Yan, Bencheng Liao, Tianyi Yan, Kaixin Xiong, Long Chen, Hongwei Xie, Bing Wang, Guang Chen, Hangjun Ye, Wenyu Liu, Haiyang Sun, and Xinggang Wang. UniDriveVLA: Unifying understanding, perception, and action planning for autonomous driving. *arXiv preprint arXiv:2604.02190*, 2026.
- [2] Ellington Kirby, Alexandre Boulch, Yihong Xu, Yuan Yin, Gilles Puy, Éloi Zablocki, Andrei Bursuc, Spyros Gidaris, Renaud Marlet, Florent Bartoccioni, Anh-Quan Cao, Nermin Samet, Tuan-Hung Vu, and Matthieu Cord. Driving on registers. *arXiv preprint arXiv:2601.05083*, 2026.
- [3] Holger Caesar, Varun Bankiti, Alex H. Lang, et al. nuScenes: A multimodal dataset for autonomous driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [4] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, et al. NAV-SIM: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.15349.
- [5] Wei Cao, Marcel Hallgarten, Tianyu Li, Daniel Dauner, et al. Pseudo-simulation for autonomous driving. In *Conference on Robot Learning (CoRL)*, 2025.
- [6] Yihan Hu, Jiazhi Yang, Li Chen, et al. Planning-oriented autonomous driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [7] Zhenxin Li, Kailin Li, Shihao Wang, Shiyi Lan, Zhiding Yu, et al. Hydra-MDP: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*, 2024.
- [8] Wenhao Yao, Zhenxin Li, Shiyi Lan, Zi Wang, Xinglong Sun, Jose M. Alvarez, and Zuxuan Wu. DriveSuprim: Towards precise trajectory selection for end-to-end planning. *arXiv preprint arXiv:2506.06659*, 2025.
- [9] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *Conference on Robot Learning (CoRL)*, 2023.
- [10] Bo Jiang, Shaoyu Chen, Qing Xu, et al. VAD: Vectorized scene representation for efficient autonomous driving. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

- [11] Wenchao Sun, Xuewu Lin, Yining Shi, Chuang Zhang, Haoran Wu, and Sifa Zheng. SparseDrive: End-to-end autonomous driving via sparse scene representation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [12] Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, et al. TransFuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2023.
- [13] Yuning Chai, Benjamin Sapp, Mayank Bansal, and Dragomir Anguelov. Multi-Path: Multiple probabilistic anchor trajectory hypotheses for behavior prediction. In *Conference on Robot Learning (CoRL)*, 2019.
- [14] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [15] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- [16] Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, and Xinggang Wang. DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [17] Abner Guzman-Rivera, Dhruv Batra, and Pushmeet Kohli. Multiple choice learning: Learning to produce multiple structured outputs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [18] Bo Jiang, Shaoyu Chen, Hao Gao, Bencheng Liao, Qian Zhang, Wenyu Liu, and Xinggang Wang. VADv2: End-to-end vectorized autonomous driving via probabilistic planning. In *International Conference on Learning Representations (ICLR)*, 2026.
- [19] Zhenxin Li, Wenhao Yao, Zi Wang, Xinglong Sun, Joshua Chen, Nadine Chang, Maying Shen, Zuxuan Wu, Shiyi Lan, and Jose M. Alvarez. Generalized trajectory scoring for end-to-end multimodal planning. *arXiv preprint arXiv:2506.06664*, 2025.
- [20] Zhenxin Li, Wenhao Yao, Zi Wang, et al. ZTRS: Zero-imitation end-to-end autonomous driving with trajectory scoring. *arXiv preprint arXiv:2510.24108*, 2025.
- [21] Xiaoyu Tian, Junru Gu, Bailin Li, et al. DriveVLM: The convergence of autonomous driving and large vision-language models. In *Conference on Robot Learning (CoRL)*, 2024.
- [22] Zewei Zhou, Tianhui Cai, Seth Z. Zhao, Yun Zhang, Zhiyu Huang, Bolei Zhou, and Jiaqi Ma. AutoVLA: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. *arXiv preprint arXiv:2506.13757*, 2025.
- [23] Yongkang Li, Kaixin Xiong, Xiangyu Guo, Fang Li, Sixu Yan, Gangwei Xu, Lijun Zhou, Long Chen, Haiyang Sun, Bing Wang, et al. ReCogDrive: A reinforced cognitive framework for end-to-end autonomous driving. *arXiv preprint arXiv:2506.08052*, 2025.

- [24] Peiru Zheng, Yun Zhao, Zhan Gong, Hong Zhu, and Shaohua Wu. SimpleVSF: VLM-scoring fusion for trajectory prediction of end-to-end autonomous driving. *arXiv preprint arXiv:2510.17191*, 2025.
- [25] Zilin Wang, Saeed Rahmani, Daphne Cornelisse, Bidipta Sarkar, Alexander David Goldie, Jakob Nicolaus Foerster, and Shimon Whiteson. Learning to drive in new cities without human demonstrations. *arXiv preprint arXiv:2602.15891*, 2026.
- [26] Shuai Bai et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [27] Tiankai Hang, Shuyang Gu, Chen Li, Jianmin Bao, Dong Chen, Han Hu, Xin Geng, and Baining Guo. Efficient diffusion training via min-SNR weighting strategy. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [28] Christian Rupprecht, Iro Laina, Robert DiPietro, Maximilian Baust, Federico Tombari, Nassir Navab, and Gregory D. Hager. Learning in an uncertain world: Representing ambiguity through multiple hypotheses. In *IEEE International Conference on Computer Vision (ICCV)*, 2017.